UNIVERSITY OF MINNESOTA
Driven to Discover®

Runtime Engine Architecture

CSCI 5980: Game Engine Architecture

Evan Suma Rosenberg | CSCI 5980 | Spring 2026

Today

- Overview of runtime engine architecture
- Hardware, drivers, and operating system layers
- Third-party SDKs and middleware
- Platform independence layer
- Core systems
- Rendering engine architecture
- Physics, animation, audio, and input systems
- Gameplay foundation systems

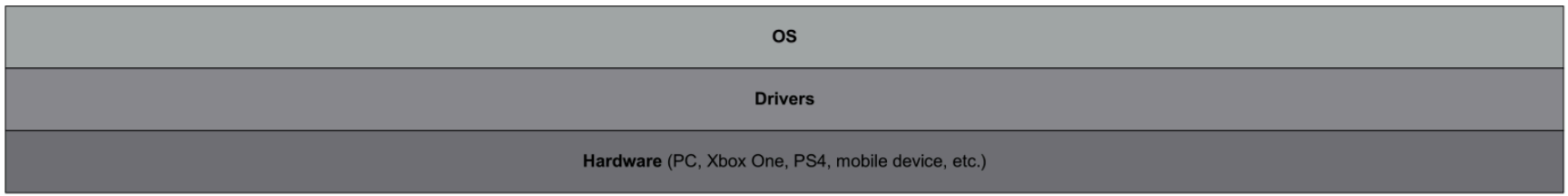
Game Engine Components

- A game engine consists of:
  - Tool suite: content creation and asset pipeline
  - Runtime component: executes the game in real-time
- This lecture focuses on the runtime architecture
- Game engines are large software systems
  - Built in layers with dependencies flowing downward
  - Upper layers depend on lower layers, not vice versa

Layered Architecture

- Dependency direction matters
  - Upper layers depend on lower layers
  - Lower layers should NOT depend on higher layers
- Circular dependencies should be avoided
  - Lead to undesirable coupling between systems
  - Make software untestable
  - Inhibit code reuse
- Especially important in large-scale systems like game engines

Hardware and System Layers



Target Hardware

- Typical hardware platforms include:
  - PC: Windows, Linux, MacOS
  - Mobile: iOS, Android, Kindle Fire
  - Consoles: Xbox, PlayStation, Nintendo Switch
- Most topics in this class are platform-agnostic
- Some design considerations are specific to PC vs console development

Device Drivers

- Low-level software components provided by:
  - Operating system
  - Hardware vendor
- Purpose:
  - Manage hardware resources
  - Shield upper layers from hardware communication details
  - Handle the myriad variants of available hardware devices

Operating System

- On PC:
  - OS runs continuously, orchestrating multiple programs
  - Uses preemptive multitasking (time-sliced sharing)
  - Games cannot assume full hardware control
  - Must "play nice" with other programs
- On early consoles:
  - OS was a thin library compiled into the game executable
  - Game "owned" the entire machine

Modern Console Operating Systems

- Modern consoles (Xbox One, PS4) have more complex OS behavior:
  - Can interrupt game execution
  - Take over system resources for online messages
  - Allow player to pause and access system UI
- Background tasks run continuously:
  - Recording gameplay video for sharing
  - Downloading games, patches, and DLC
- Gap between console and PC development is gradually closing

Third-Party SDKs

- Most game engines leverage third-party software:
  - SDKs (Software Development Kits)
  - Middleware packages
- The interface provided by an SDK is called an API (Application Programming Interface)

Data Structures and Algorithms Libraries

- Boost: Powerful library designed in the style of C++ standard library
  - Great documentation for learning computer science concepts
- Folly: Facebook's library extending standard C++ and Boost
  - Emphasis on maximizing code performance
- Loki: Generic programming template library
  - Powerful but complex

C++ Standard Library and STL

- Standard library provides similar facilities to third-party libraries
- STL (Standard Template Library):
  - Generic container classes: `std::vector`, `std::list`
  - Originally written by Alexander Stepanov and David Musser
  - Much functionality absorbed into C++ standard library
- "STL" typically refers to the container subset of the standard library

Graphics SDKs

- Glide: SDK for old Voodoo graphics cards
  - Pre-hardware transform, clipping, and lighting era
- OpenGL: Widely used portable 3D graphics SDK
- DirectX: Microsoft's 3D graphics SDK, primary rival to OpenGL
- Vulkan: Low-level library by Khronos Group
  - Submit rendering batches and compute jobs directly to GPU
  - Fine-grained control over memory and CPU/GPU shared resources

Console Graphics SDKs

- Edge: Powerful rendering and animation engine by Naughty Dog/Sony
  - Used by first and third-party game studios on PlayStation 3

- libgcm: Low-level interface to PlayStation 3's RSX graphics hardware
  - More efficient alternative to OpenGL on PS3

Collision and Physics SDKs

- Havok: Popular industrial-strength physics and collision engine
  - Gold standard in the industry
- PhysX: NVIDIA's physics and collision engine
  - Available for free download
  - Originally designed for Ageia's physics accelerator chip
  - Now runs on NVIDIA GPUs
- Open Dynamics Engine (ODE)
  - Open source physics/collision package

Character Animation SDKs

- Granny (Rad Game Tools):
  - 3D model and animation exporters for Maya, 3DS Max, etc.
  - Runtime library for reading/manipulating animation data
  - Excellent handling of time in animation API
- Havok Animation:
  - Complements Havok physics SDK
  - Bridges the physics-animation gap

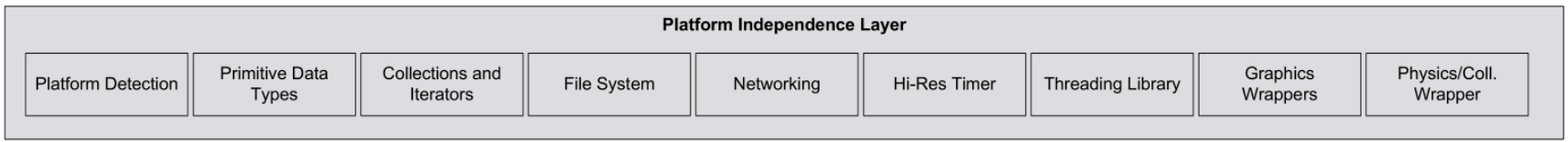
Character Animation SDKs (Console)

- OrbisAnim (PS4):
  - Produced by SN Systems with Naughty Dog and Sony teams
  - Powerful and efficient animation engine
  - Efficient geometry-processing for rendering

Biomechanical Character Models

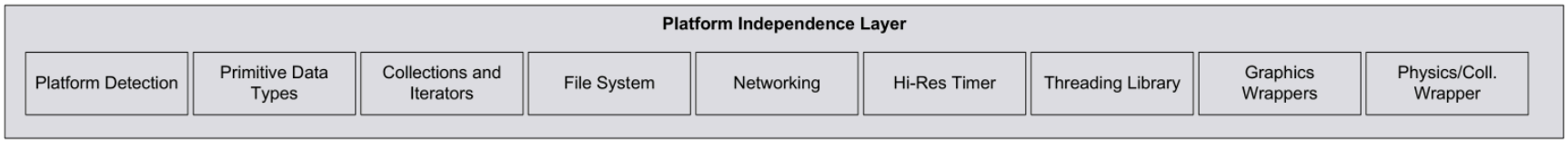
- Endorphin (Natural Motion):
  - Maya plug-in for biomechanical simulations
  - Accounts for center of gravity, weight distribution
  - Models how humans balance and move under gravity/forces
  - Export results as hand-animated data
- Euphoria:
  - Real-time version of Endorphin
  - Produces physically accurate motion at runtime
  - Responds to unpredictable forces

Platform Independence Layer



- Most game engines must run on multiple hardware platforms
  - Exposes games to the largest possible market
- Only exception: first-party studios (e.g., Naughty Dog, Insomniac)
  - May target only one platform

Platform Independence Layer

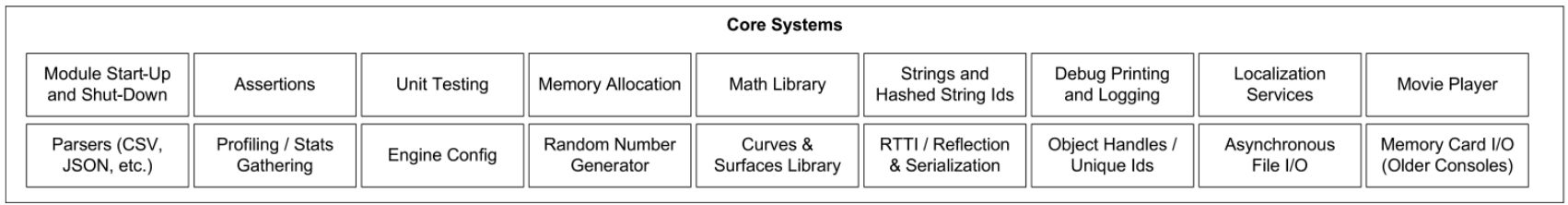


- Sits atop hardware, drivers, OS, and third-party software
- Shields rest of engine from platform-specific details
- "Wraps" functionality to provide consistent interface across platforms

Why Wrap Functions?

- Reason 1: API inconsistency
  - Some APIs (OS functions, C standard library) differ significantly between platforms
  - Wrapping provides a consistent API across all target platforms
- Reason 2: Future-proofing
  - Even with cross-platform libraries (e.g., Havok)
  - Insulate yourself from future changes
  - Easier to transition to different libraries later

Core Systems



- Every large C++ software application needs utility facilities
- The core systems layer provides these fundamental services
- Not game-specific, but essential for engine operation

Assertions

- Lines of error-checking code inserted to catch:
  - Logical mistakes
  - Violations of programmer's original assumptions
- Stripped out of final production build
- Essential for debugging during development

Memory Management

- Virtually every game engine implements **custom memory allocation**
- Goals:
  - High-speed allocations and deallocations
  - Limit negative effects of memory fragmentation
- Standard allocators often too slow or unpredictable for real-time use

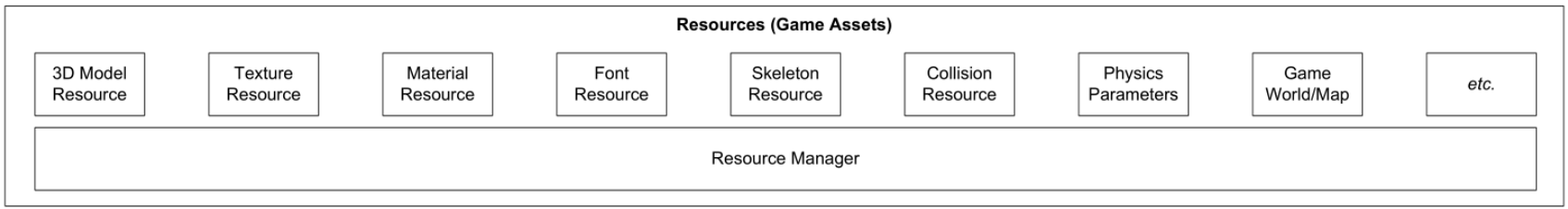
Math Library

- Games are **highly mathematics-intensive**
- Math libraries provide:
  - Vector and matrix math
  - Quaternion rotations
  - Trigonometry
  - Geometric operations (lines, rays, spheres, frusta)
  - Spline manipulation
  - Numerical integration
  - Solving systems of equations

Custom Data Structures and Algorithms

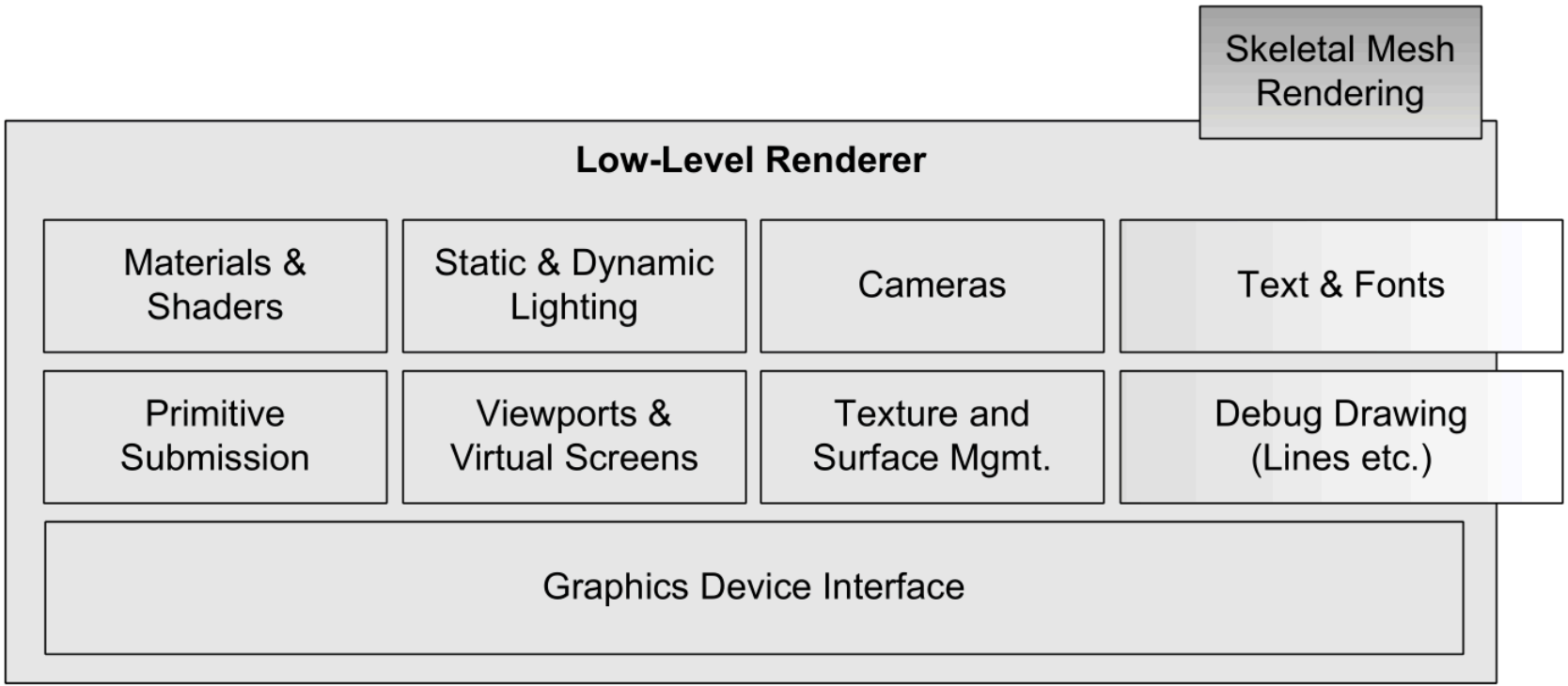
- Unless relying entirely on third-party packages (Boost, Folly):
  - Custom implementations are usually required
- Data structures: linked lists, dynamic arrays, binary trees, hash maps
- Algorithms: search, sort, etc.
- Often hand-coded to:
  - Minimize or eliminate dynamic memory allocation
  - Ensure optimal runtime performance on target platforms

Resource Manager



- Present in every game engine in some form
- Provides unified interface for accessing game assets
- Implementation approaches vary:
  - Centralized and consistent: Unreal's packages, OGRE's ResourceManager
  - Ad hoc: Direct file access, compressed archives (Quake's PAK files)

Rendering Engine



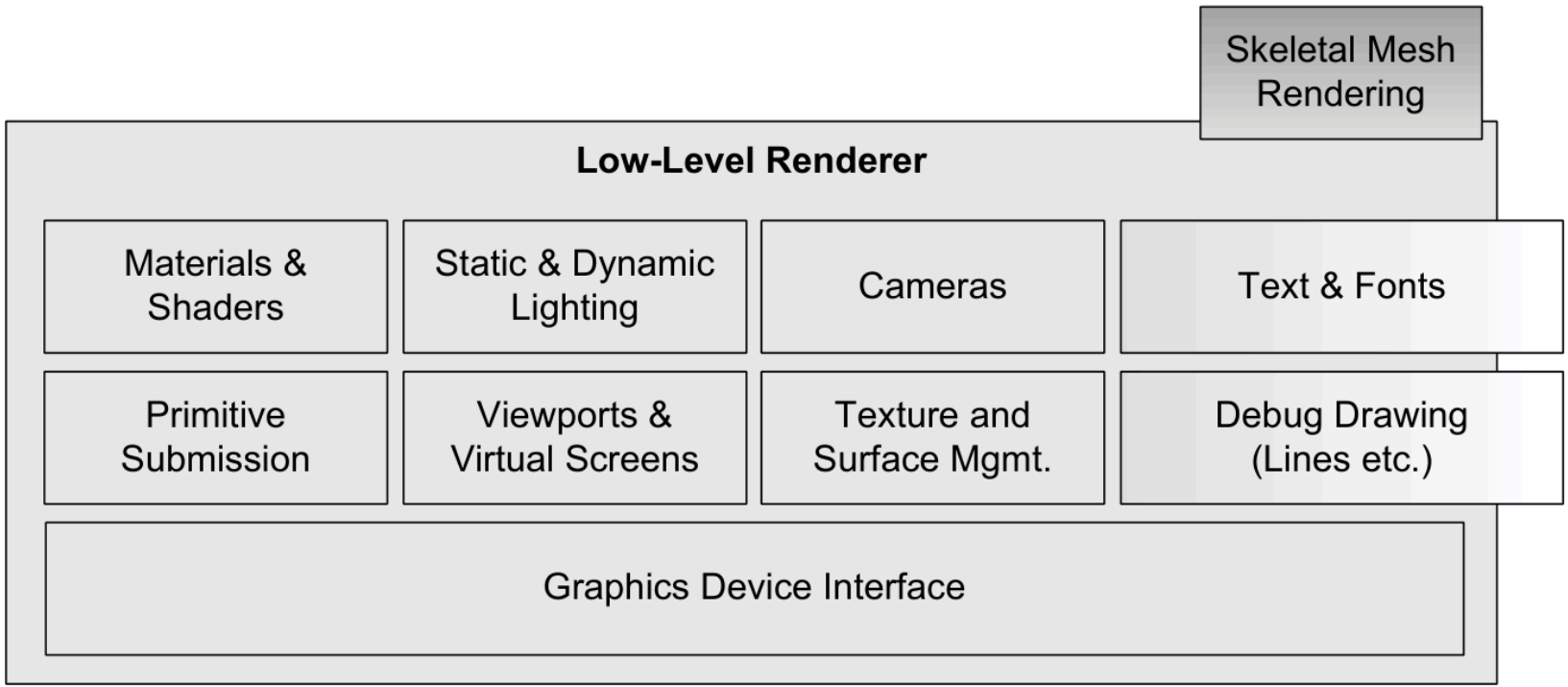
Rendering Engine

- One of the largest and most complex components of any game engine
- Can be architected in many different ways
  - No single accepted approach
- Modern renderers share fundamental design philosophies
  - Driven by 3D graphics hardware design
- Common approach: layered architecture

Low-Level Renderer

- Encompasses all raw rendering facilities of the engine
- Design focus:
  - Render geometric primitives as quickly and richly as possible
  - Without much regard for visibility determination

Low-Level Renderer



Graphics Device Interface

- Graphics SDKs (DirectX, OpenGL, Vulkan) require setup code:
  - Enumerate available graphics devices
  - Initialize them
  - Set up render surfaces (back-buffer, stencil buffer, etc.)
- For PC games:
  - Integrate renderer with Windows message loop
  - Write a "message pump" to service Windows messages
  - Run render loop when no messages pending

Other Low-Level Renderer Components

- Collect submissions of geometric primitives (render packets):
  - Meshes, line lists, point lists
  - Particles, terrain patches
  - Text strings
- Viewport abstraction:
  - Camera-to-world matrix
  - 3D projection parameters (FOV, near/far clip planes)

Materials and Lighting

- Material system manages:
  - Graphics hardware state
  - Game shaders
  - Texture(s) used by primitives
  - Vertex and pixel shader selection
- Dynamic lighting system:
  - Each primitive affected by n dynamic lights
  - Determines how lighting calculations are applied

Scene Graph / Culling Optimizations

- Low-level renderer draws **all submitted geometry**
  - Only basic back-face culling and frustum clipping
- Higher-level component needed to limit primitives based on **visibility**
- For small game worlds:
  - Simple **frustum cull** may be sufficient
- For larger worlds:
  - Advanced **spatial subdivision** data structures

Spatial Subdivision Structures

- Improve rendering efficiency by quickly determining potentially visible set (PVS)
- Types include Binary Space Partitioning (BSP) tree, quadtree, octree, kd-tree, sphere hierarchy
- Sometimes called a "scene graph" (though technically different)
- Portals or occlusion culling may also be applied

Decoupling Scene Graph from Renderer

- Ideally, low-level renderer should be agnostic to spatial subdivision type
- Benefits:
  - Different game teams can reuse primitive submission code
  - Craft PVS determination specific to each game's needs
- Example: OGRE provides plug-and-play scene graph architecture
  - Select from pre-implemented designs
  - Or provide custom implementation

Visual Effects

- Modern engines support a wide range of effects:
  - Particle systems: smoke, fire, water splashes
  - Decal systems: bullet holes, footprints
  - Light mapping and environment mapping
  - Dynamic shadows
  - Full-screen post effects: applied after 3D scene rendering

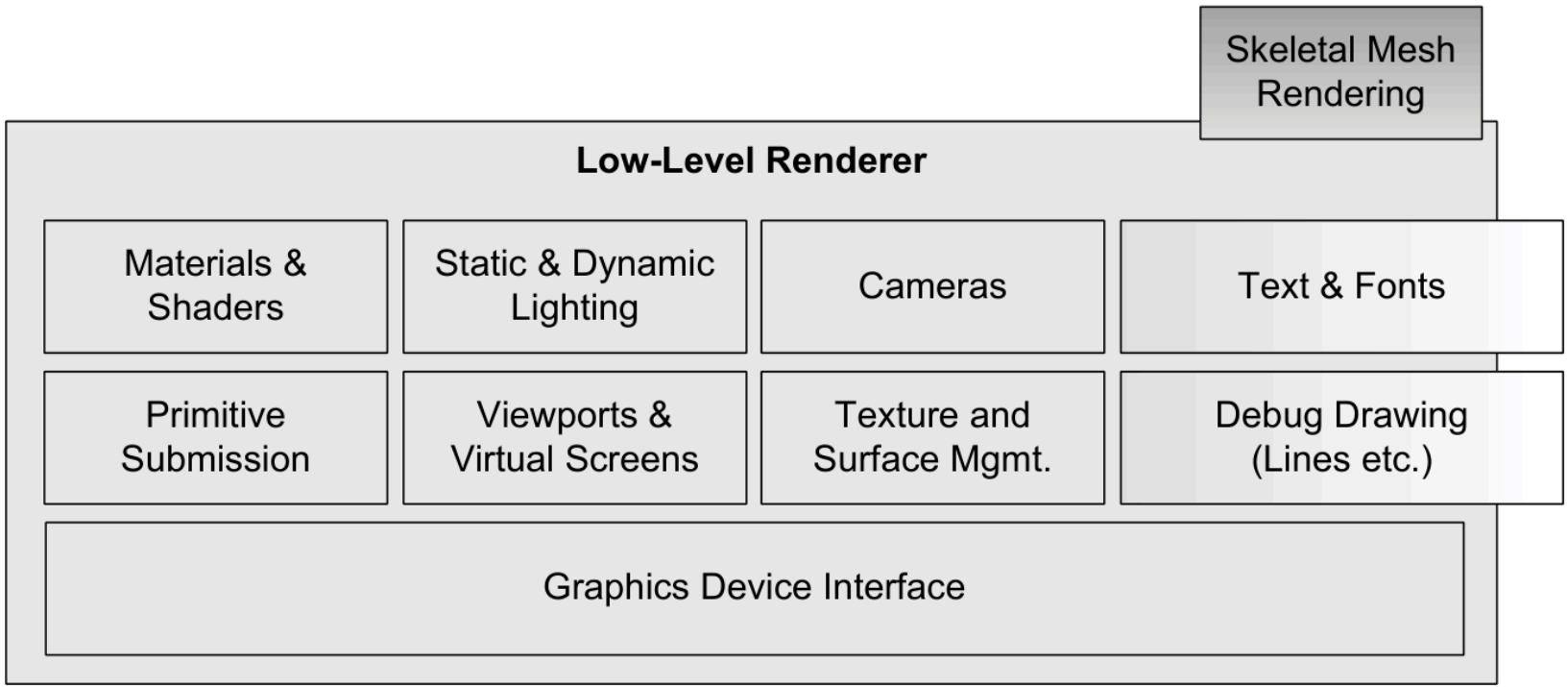
Full-Screen Post Effects

- Applied after 3D scene rendered to off-screen buffer
- Examples:
  - HDR (High Dynamic Range) tone mapping and bloom
  - FSAA (Full-Screen Anti-Aliasing)
  - Color correction and color-shift effects
  - Bleach bypass, saturation/desaturation

Effects System Architecture

- **Effects system** component often manages specialized rendering needs
- **Particle and decal systems**:
  - Act as inputs to low-level renderer
- **Light mapping, environment mapping, shadows**:
  - Handled internally within renderer
- **Full-screen post effects**:
  - Integral part of renderer OR separate component operating on output buffers

Front End Graphics



Front End Graphics

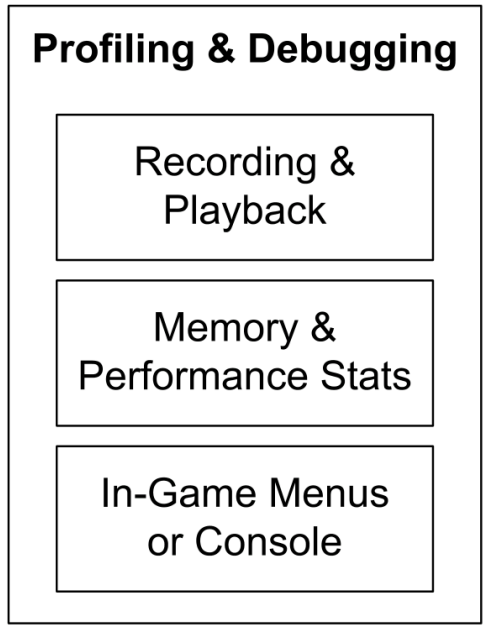
- 2D graphics overlaid on 3D scene:
  - HUD (Heads-Up Display)
  - In-game menus, console, development tools
  - In-game GUI for inventory, unit configuration, etc.
- Implementation:
  - Textured quads with orthographic projection
  - Or full 3D with bill-boarded quads facing camera

Full-Motion Video (FMV)

- System for playing **pre-recorded full-screen movies**
  - Rendered with game engine or external rendering package
- **In-Game Cinematics (IGC)**:
  - Choreograph cinematic sequences within the game
  - Done in full 3D at runtime (no pre-rendered movies)
- Example: Uncharted 4 displays all cinematics as real-time IGCs

Profiling and Debugging Tools

- Games are **real-time systems**
  - Must profile performance to optimize
- Memory resources are usually scarce
  - Heavy use of memory analysis tools
- Layer also includes:
  - Debug drawing facilities
  - In-game menu/console system
  - Gameplay recording and playback



General-Purpose Profiling Tools

- Intel VTune
- IBM Quantify and Purify (PurifyPlus tool suite)
- Insure++ by Parasoft
- Valgrind by Julian Seward

Custom Engine Profiling Tools

- **Manual code instrumentation**: Time specific code sections
- **On-screen profiling display**: Stats while game runs
- **Performance stat export**: Text files or Excel spreadsheets
- **Memory usage tracking**: Per-subsystem memory consumption
- **Memory dump facilities**: High water mark and leak detection

Custom Engine Debugging Tools

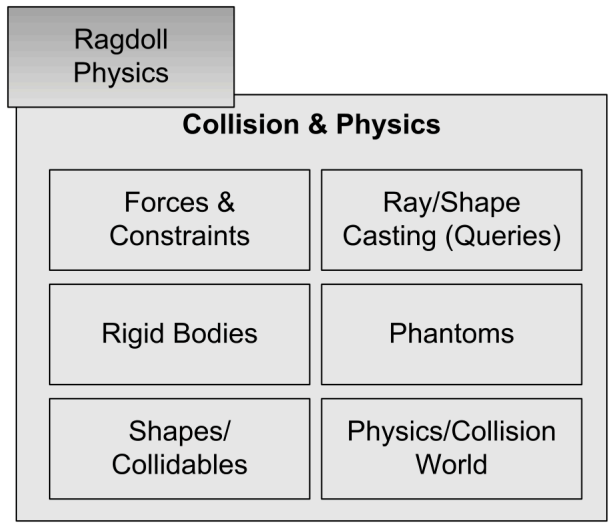
- **Debug print statements**: Peppered throughout code
  - Category-based enable/disable
  - Verbosity level control
- **Event recording and playback**:
  - Record game events, replay for debugging
  - Tough to implement correctly
  - Very valuable for tracking down bugs

PlayStation 4 Core Dump Facility

- Powerful debugging aid for crashes
- Automatically provides:
  - Complete call stack at crash
  - Screenshot of crash moment
  - 15 seconds of video footage before crash
- Core dumps can be automatically uploaded to developer servers
  - Works even after game has shipped
  - Revolutionizes crash analysis and repair

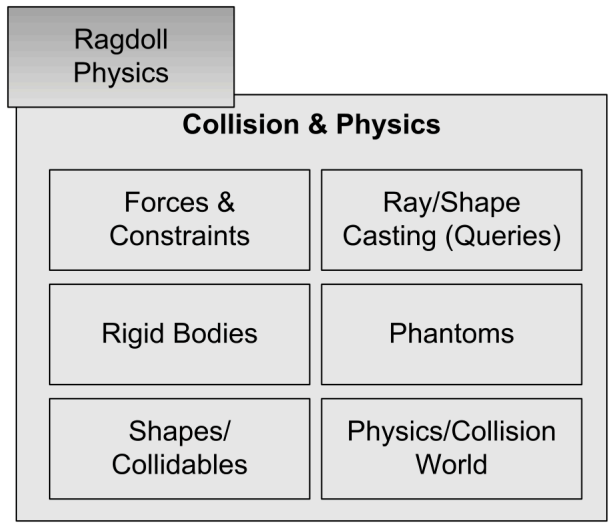
Collision and Physics

- **Collision detection** is important for every game
  - Without it, objects would interpenetrate
  - Impossible to interact with virtual world
- **Physics system** (rigid body dynamics)
  - Realistic or semi-realistic dynamics simulation
  - Motion (kinematics) of rigid bodies
  - Forces and torques causing motion



Collision and Physics Coupling

- Collision and physics are **tightly coupled**
- Collisions are almost always resolved as part of physics integration
- Very few companies write their own collision/physics engine
- Third-party SDK typically integrated



Animation

- Five basic types used in games:
  - Sprite/texture animation
  - Rigid body hierarchy animation
  - Skeletal animation
  - Vertex animation
  - Morph targets
- Skeletal animation is the most prevalent method today

Skeletal Animation

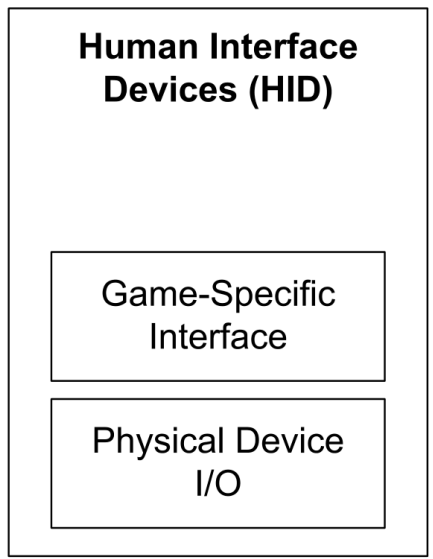
- 3D character mesh posed using a system of **bones**
- As bones move, mesh vertices move with them
- Animation system produces:
  - Pose for every bone in skeleton
  - Passed to renderer as **palette of matrices**
- Renderer transforms vertices by matrices
  - Process called **skinning**

Rag Doll Physics

- Rag doll: Limp (often dead) animated character
- Motion simulated by physics system:
  - Treats body parts as constrained rigid bodies
  - Determines positions and orientations
- Animation system calculates matrices for rendering
- Creates tight coupling between animation and physics systems

Human Interface Devices

- Every game needs to process player input
  - Keyboard and mouse
  - Game controllers and joysticks
  - Specialized controllers (steering wheels, dance pads, motion controls, etc.)
- Also called player I/O component:
  - May provide output (force-feedback, rumble)
  - Audio from devices (controller speaker)



HID Engine Component

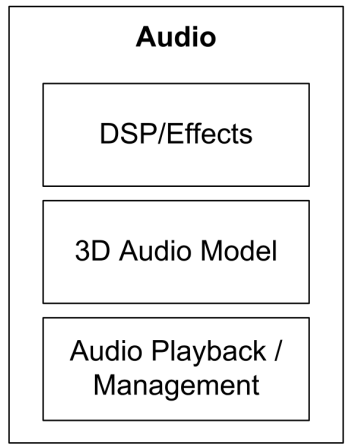
- Divorces low-level hardware details from high-level game controls
- Input processing:
  - Dead zone around joypad stick center
  - Button debouncing
  - Button-down and button-up event detection
  - Accelerometer smoothing (PlayStation Dualshock)

HID Engine Component Features

- **Control remapping**: Player customizes button-to-function mapping
- **Chord detection**: Multiple buttons pressed together
- **Sequence detection**: Buttons pressed in order within time limit
- **Gesture recognition**: Sequences from buttons, sticks, accelerometers

Audio

- Audio is just as important as graphics in any game engine
- Unfortunately, audio often gets less attention than:
  - Rendering, physics, animation, AI, gameplay
- Programmers often develop with speakers off!
- No great game is complete without a stunning audio engine



Audio Engines

- Audio engines vary greatly in sophistication
- Quake's audio: Pretty basic, teams often augment or replace it
- Unreal Engine 4: Reasonably robust 3D audio rendering
  - Limited feature set, teams may want to customize
- XAudio2 (Microsoft): Excellent runtime audio for DirectX platforms
- SoundR!OT (EA): Advanced in-house engine
- Scream (Sony/Naughty Dog): Powerful 3D audio for PS3/PS4 titles

Audio in Game Development

- Even with a preexisting audio engine, games require:
  - Significant custom software development
  - Integration work
  - Fine-tuning and attention to detail
- All necessary to produce high-quality audio in the final product

Multiplayer (Offline)

- **Single-screen multiplayer**:
  - Multiple HIDs connected to one machine
  - Single camera keeps all players in frame
  - Examples: Smash Brothers, Lego Star Wars, Gauntlet
- **Split-screen multiplayer**:
  - Multiple players, each with own camera
  - Screen divided into sections

Multiplayer (Online)

- **Networked multiplayer**:
  - Multiple computers/consoles networked
  - Each machine hosts one player
- **Massively Multiplayer Online Games (MMOG)**:
  - Hundreds of thousands of simultaneous users
  - Giant, persistent online virtual world
  - Hosted by powerful central servers

Multiplayer Impact on Engine Design

- Support for multiple players has **profound impact** on engine design:
  - Game world object model
  - Renderer
  - Human input device system
  - Player control system
  - Animation systems
- Retrofitting multiplayer into a single-player engine is daunting
- Better to design multiplayer features from **day one**

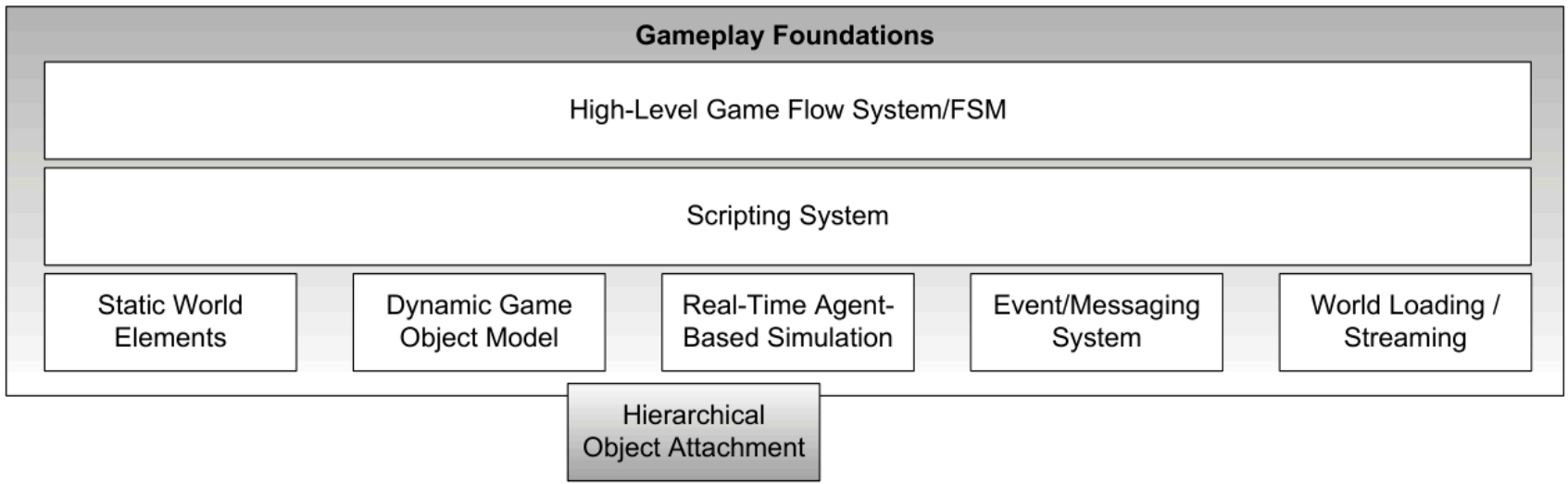
Single-Player as Multiplayer Special Case

- Converting multiplayer to single-player is typically **trivial**
- Many engines treat single-player as special case:
  - Multiplayer game with only one player
- Example: **Quake engine's client-on-top-of-server mode**
  - Single executable on single PC
  - Acts as both client and server in single-player campaigns

Gameplay Foundation Systems

- Gameplay: the action in the game
  - Rules governing the virtual world
  - Player character abilities (player mechanics)
  - Other characters and objects
  - Goals and objectives
- Can be implemented in:
  - Native language (C++)
  - High-level scripting language

Gameplay Foundations Layer



- Bridges gap between gameplay code and low-level engine systems

Game Worlds and Object Models

- Introduces the notion of a **game world**:
  - Static elements (background geometry)
  - Dynamic elements (moving objects)
- Contents usually modeled in **object-oriented manner**
- **Game object model**: collection of object types in the game
  - Real-time simulation of heterogeneous objects

Typical Game Object Types

- Static background geometry: buildings, roads, terrain
- Dynamic rigid bodies: rocks, soda cans, chairs
- Player characters (PC)
- Non-player characters (NPC)
- Weapons
- Projectiles
- Vehicles
- Lights (runtime or offline only)

Software Object Model

- Game world model is tied to software object model
  - Object-oriented design approach?
  - Programming language choice (C, C++, Java)?
  - Class hierarchy organization?
  - Templates/policy-based design vs. polymorphism?
  - How are objects referenced?
    - Pointers, smart pointers, handles?
  - How are objects uniquely identified?
    - Memory address, name, GUID?
  - How are object lifetimes managed?

Event System

- Game objects need to **communicate** with each other
- Approaches:
  - Direct member function calls on receiver object
  - **Event-driven architecture** (like GUI systems)
- Event-driven system:
  - Sender creates **event/message** data structure
  - Contains message type and argument data
  - Passed to receiver's event handler function
  - Events can be queued for future handling

Scripting System

- Many engines use scripting language for:
  - Game-specific gameplay rules
  - Easier and more rapid content development
- Without scripting:
  - Must recompile/relink executable for every change
- With scripting:
  - Modify and reload script code
  - Some engines allow reload while game runs
  - Much faster turnaround time

Artificial Intelligence Foundations

- Traditionally, AI was game-specific software
  - Not considered part of engine
- Now, companies recognize common patterns in AI systems
- Low-level AI building blocks becoming part of engine:
  - Nav mesh generation
  - Path finding
  - Static and dynamic object avoidance
  - Vulnerability identification
  - AI-animation interface

AI Middleware

- Kynapse (Kynogon)
  - Early AI middleware SDK
- Gameware Navigation (Autodesk):
  - Redesigned by same engineering team
  - Provides low-level AI building blocks
  - Well-defined interface between AI and animation

Game-Specific Subsystems

- Built on top of gameplay foundations and low-level engine
- Gameplay programmers and designers cooperate to implement:
  - Features specific to the game being developed
- Usually numerous, highly varied, and game-specific

Examples of Game-Specific Subsystems

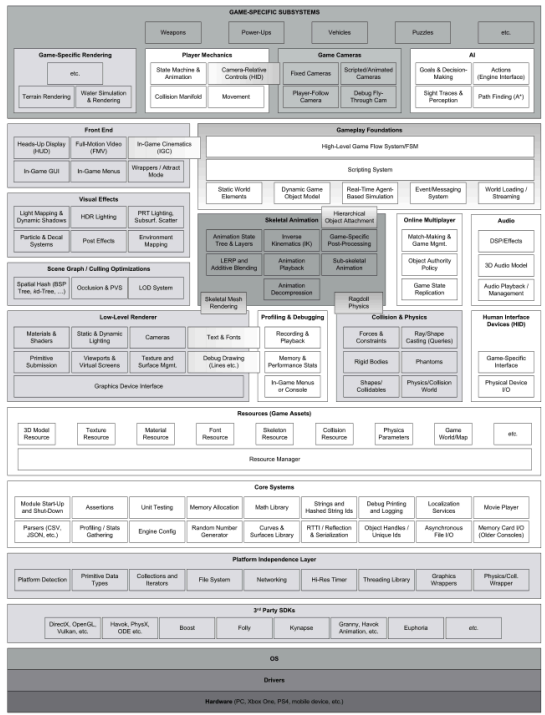
- Player mechanics: character abilities and controls
- Game cameras: player-follow, fixed, scripted, debug fly-through
- AI for NPCs: goals, decision-making, perception, path finding
- Weapon systems
- Vehicles
- Power-ups
- Puzzles
- And many more...

Engine vs. Game Boundary

- If a clear line could be drawn between engine and game, it would lie between game-specific subsystems and gameplay foundations
- Practically speaking, the line is never perfectly distinct
- Some game-specific knowledge inevitably:
  - Seeps down through gameplay foundations
  - Sometimes extends into core of engine itself

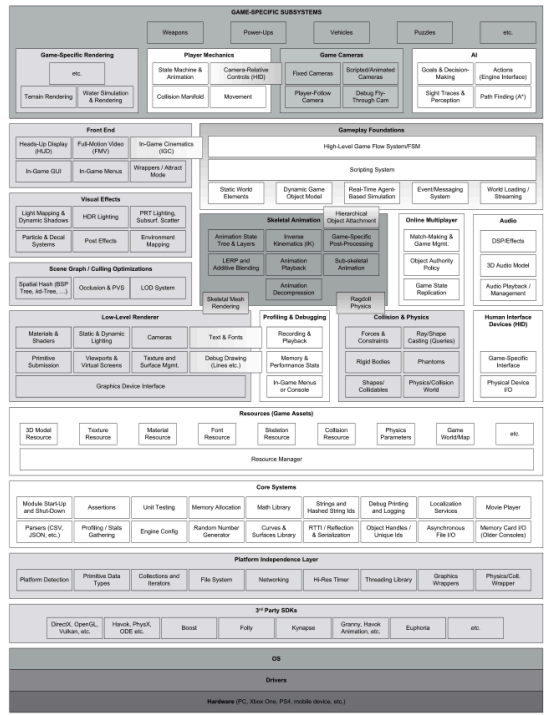
Key Takeaways

- Game engines are layered architectures
  - Dependencies flow from upper to lower layers
  - Avoid circular dependencies
- Lower layers provide foundation:
  - Hardware, drivers, OS, third-party SDKs
  - Platform independence layer
  - Core systems



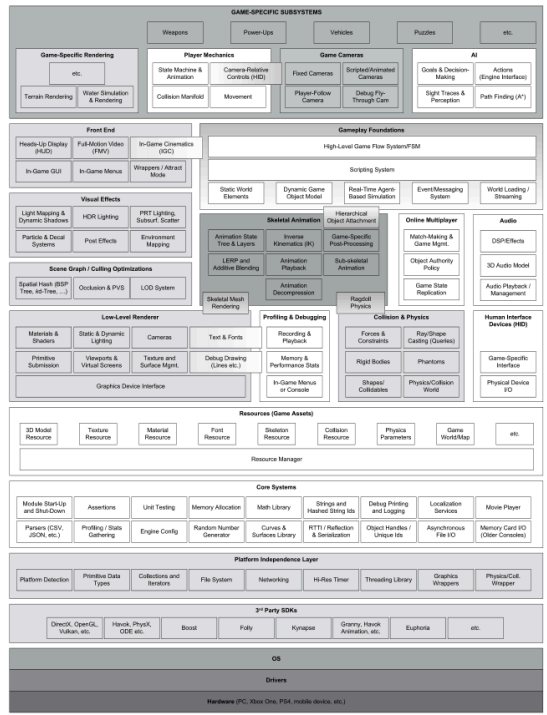
Key Takeaways

- Middle layers handle major subsystems:
  - Resource management
  - Rendering engine (low-level, scene graph, visual effects, front end)
  - Physics and collision
  - Animation
  - Audio
  - Human interface devices
  - Networking/multiplayer



Key Takeaways

- Upper layers build game-specific functionality:
  - Gameplay foundations (object model, events, scripting, AI)
  - Game-specific subsystems (mechanics, cameras, weapons, etc.)
- The boundary between engine and game is never perfectly distinct



Participation Exercise

- Choose one major subsystem from the runtime engine architecture
  - Renderer, physics, resource management, gameplay foundations, etc.
- Research how a specific game engine implements that subsystem.
  - Unreal, Unity, Godot, etc.
- Write a brief description (one paragraph) of the subsystem design and notable features.
- If you use AI for research, make sure to **fact check** all claims for hallucinations and inaccuracies.
- Submit your answer on Canvas within one week.

Questions?